

3.3.4 集思广益

集思广益模式强调的是通过**汇集多种不同的观点和方法**来优化决策过程。在这种模式下，模型不仅仅依赖于单一的推理路径，而是通过探索多种可能的解决方案，从中选择最优的答案。这种方法借鉴了集体智慧的概念，即通过整合多个独立的思考结果，可以得到更全面、更准确的结论。在集体智慧的启发下，研究者在 CoT 的基础上探讨了如何通过自洽性来增强模型的推理能力，提出了 Self-Consistency [36] 方法。其引入多样性的推理路径并从中选择最一致的答案，从而提高了模型的推理准确性。Self-Consistency 不依赖于特定的 CoT 形式，可以与其他 CoT 方法兼容，共同作用于模型的推理过程。

如图 3.15 所示，Self-Consistency 的实现过程可以分为三个步骤：(1) 在随机采样策略下，使用 CoT 或 Zero-Shot CoT 的方式来引导大语言模型针对待解决问题生成一组多样化的推理路径；(2) 针对大语言模型生成的每个推理内容，收集其最终的答案，并统计每个答案在所有推理路径中出现的频率；(3) 选择出现频率最高的答案作为最终的、最一致的答案。

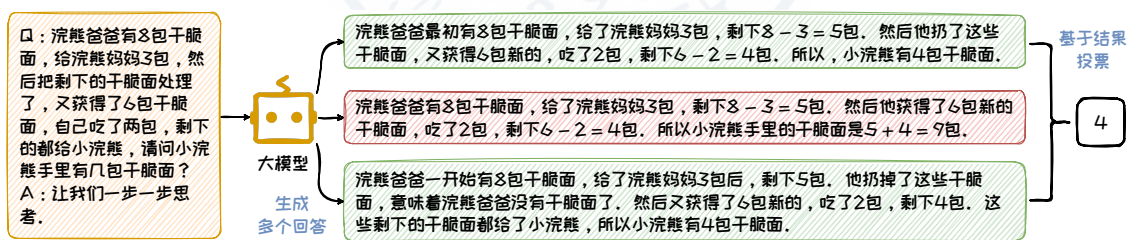


图 3.15: Self-Consistency 的流程。

本节探讨了思维链（Chain-of-Thought, CoT）的思想和模式。CoT 的核心思想是在提示中模拟人类解决问题的思考过程，在 Prompt 中嵌入解决问题的推理过程，从而在无需特定任务微调的情况下显著提升模型在推理任务上的表现。CoT 方法包含了多种模式：按部就班、按部就班以及集思广益。这些模式可以满足不同的推理需求，增强了大语言模型在复杂任务中的推理能力。

3.4 Prompt 技巧

基于上下文学习和思维链等 Prompt 工程技术，本节将进一步探讨可用于进一步提升大语言模型生成质量的 Prompt 技巧，包括合理归纳提问、适时运用思维链 (CoT) 以及巧妙运用心理暗示等。应用本节中介绍的 Prompt 技巧，可以引导模型生成更加精准、符合预期的内容，进一步提升大语言模型在实际应用中的表现。

3.4.1 规范 Prompt 编写

编写规范的 Prompt 是我们与大语言模型进行有效沟通的基础。经典的 Prompt 通常由任务说明，上下文，问题，输出格式等部分中的一个或几个组成。以图 3.16 中这个情感分类的 Prompt 为例。在这个例子中：

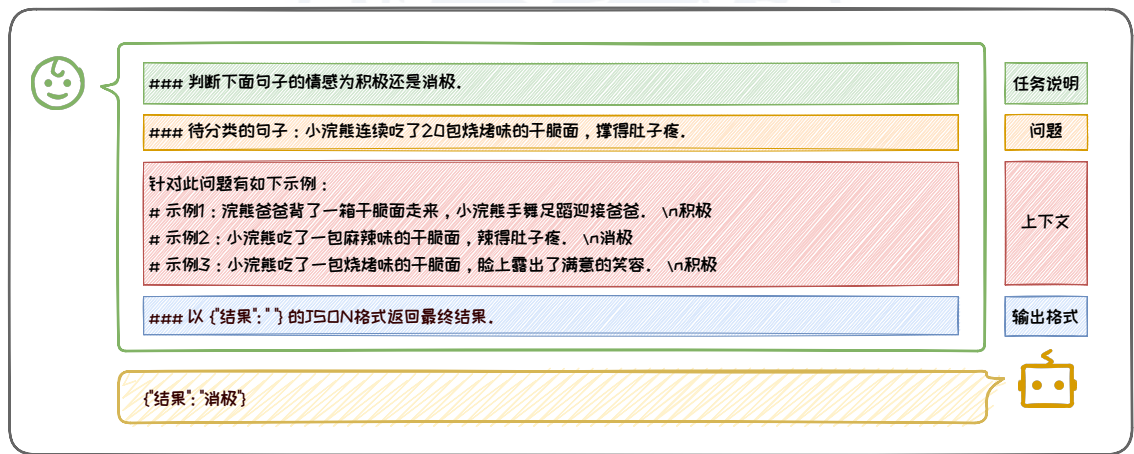


图 3.16: 经典的 Prompt 示例。

- **任务说明**是“### 判断下面句子的情感为积极还是消极。”，它明确了模型需要完成的任务；
- **上下文**是“针对此问题有如下示例：# 示例 1：浣熊爸爸背了一箱干脆面走来，小浣熊手舞足蹈迎接爸爸。 \n 积极 \n # 示例 2：小浣熊吃了一包麻辣味的干脆面，辣得肚子疼。 \n 消极 \n # 示例 3：小浣熊吃了一包烧烤味的干脆

面，脸上露出了满意的微笑。
积极”。上下文提供了帮助模型理解和回答问题的示例或背景信息；

- **问题**是“待分类的句子：小浣熊连续吃了 20 包烧烤味的干脆面，撑得肚子疼。”，是用户真正想要模型解决的问题，它可以是一个段落（比如摘要总结任务中被总结的段落），也可以是一个实际的问题（比如问答任务中用户的问题），或者表格等其他类型的输入内容；
- **输出格式**是“以”结果”：”的 JSON 格式返回最终结果。”，它规范了模型的输出格式。

通过这个例子可以看出，在编写经典 Prompt 的过程中，Prompt 各个组成部分都很重要，它们的规范性，直接影响模型的输出质量。同时，各个组成部分的排版也很重要。接下来，我们将详细介绍经典 Prompt 的规范编写需要满足的要求。

1. 任务说明要明确

明确的任务说明是构建有效 Prompt 的关键要素之一。一个清晰、具体的任务说明能够确保模型准确理解任务要求，并产生符合预期的输出。例如，在情感分类任务中，任务说明“判断下面句子的情感为积极还是消极。”就是一个明确的示例，它清晰地定义了任务类型（情感分类）和分类的具体类别（积极或消极）。相反，模糊或不明确的任务说明可能导致模型误解用户的真实意图，从而产生不符合预期的输出。如图 3.17 所示，“分类下面的句子”这样的任务说明就缺乏具体性，没有明确指出分类的类型和类别，使得模型难以准确执行任务。

为了确保任务说明的明确性，我们需要明确以下几个要点：

- **使用明确的动词**：选择能够清晰表达动作的动词，如“判断”、“分类”、“生成”等，避免使用模糊的动词如“处理”或“操作”。
- **具体的名词**：使用具体的名词来定义任务的输出或目标，例如“积极”和“消极”在情感分类任务中提供了明确的分类标准。

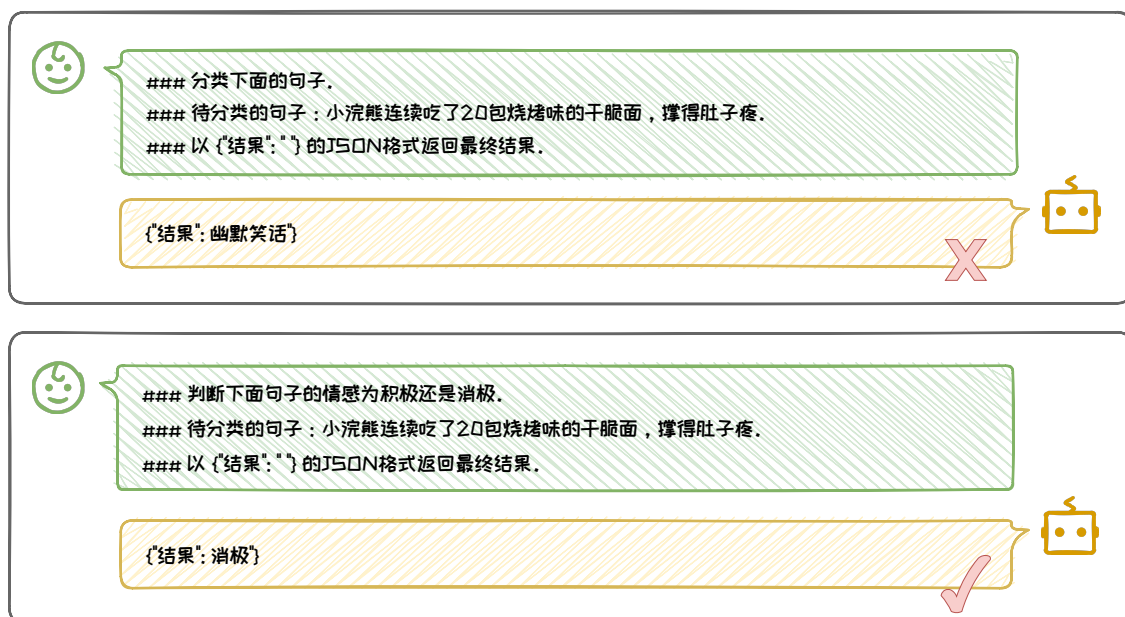


图 3.17: 不同的任务说明对比。

- **简洁明了：**任务说明应简洁且直接，避免冗长或复杂的句子结构，使模型能够快速抓住任务的核心要求。
- **结构化布局：**在较长的 Prompt 中，将任务说明放置在开头和结尾，因为模型通常更关注这些部分的信息 [18]。这种布局有助于确保模型首先和最后接触到的是最关键的任务信息。

通过这些策略，我们可以确保任务说明既清晰又具体，从而帮助模型更好地理解 and 执行任务，最终产生高质量的输出。

2. 上下文丰富且清晰

在 Prompt 设计中，上下文的作用不容忽视，它有时直接决定了模型能否给出正确的答案。一个**丰富且清晰**的上下文能够显著提升模型的理解和回答准确率。上下文的**丰富性**体现在其内容的多样性和相关性。上下文可以包括与问题直接相关的背景信息、具体的演示示例，或是对话的连续性内容。例如，在情感分类任务中，提供具体的示例句子及其对应的情感标签，可以帮助模型更好地理解任务的

具体要求和预期的输出。上下文的清晰性则要求上下文信息必须与问题紧密相关，避免包含冗余或不必要的信息。清晰的上下文应直接指向任务的核心，减少模型在处理信息时的混淆和误解。例如，在问答任务中，上下文应仅包含与问题直接相关的信息，避免引入可能误导模型的无关内容。

在图 3.18 两个上下文设计的例子中，第一个例子的上下文紧密围绕问题，提供了丰富的直接相关信息，没有任何冗余内容。这种设计有助于模型迅速聚焦于关键信息，从而准确回答问题。相比之下，第二个例子的上下文不够丰富，并且单个例子则包含了大量与问题无关的细节，这些冗余信息不仅使上下文显得不明确，还可能加重模型处理信息的负担，导致模型难以准确把握问题的核心，进而影响其回答的准确性。

判断下面句子的情感为积极还是消极。
针对此问题有如下示例：
待分类的句子：小浣熊连续吃了20包最爱的烧烤味的干脆面，撑得肚子疼，脸色憔悴。
示例1：浣熊爸爸背了一箱干脆面走来，小浣熊手舞足蹈，非常积极的迎接爸爸。\\n积极
示例2：小浣熊吃了一包烧烤味的干脆面，脸上露出了满意的笑容。\\n积极
示例3：小浣熊吃了一包麻辣味的干脆面，味道很棒，但是辣得肚子疼，它感到非常难受。\\n消极
以{"结果":""}的JSON格式返回最终结果。

{“结果”:“消极”}

判断下面句子的情感为积极还是消极的。
待分类的句子：小浣熊连续吃了20包最爱的烧烤味的干脆面，撑得肚子疼，脸色憔悴。
针对此问题有如下示例：
示例：小浣熊爸爸给小浣熊带来了一箱小浣熊最爱吃的干脆面，小浣熊兴高采烈地去迎接小浣熊爸爸。爸爸给了小浣熊一包麻辣味的干脆面，小浣熊对麻辣味的食物不太适应，但是它看出麻辣味干脆面包装很漂亮，忍不住吃了一包。麻辣味干脆面很好吃，小浣熊脸上露出了幸福的笑容。但是没过一会，它的肚子就开始疼起来了，嘴巴也火辣辣的，脸色变得苍白。\\n小浣熊高兴地迎接爸爸，并且吃了干脆面很幸福，尽管吃了面之后肚子疼，但是总体而言情感还是积极的。
以{"结果":""}的JSON形式返回最终结果。

{“结果”:“虽然小浣熊肚子疼，但是它吃到了最爱的烧烤味干脆面，所以总体而言情感是积极的”}

图 3.18: 不同的上下文对比。

3. 输出格式要规范

规范的输出格式对于确保模型输出的可用性和准确性至关重要。通过指定明确的输出格式，可以使模型的**输出结构化**，便于下游任务直接提取和使用生成内容。常用的输出格式包括 JSON、XML、HTML、Markdown 和 CSV 等，每种格式都有其特定的用途和优势。

例如，在图 3.19 中的 Prompt 例子中，“以 {”结果”: ”}”的 JSON 格式返回最终答案。”明确指定了答案应以 JSON 格式输出，并且以一个简短的例子指名 JSON 中的关键字。这种规范的输出格式不仅使得结果易于解析和处理，还提高了模型输出的准确性和一致性。如果不明确规定输出格式，模型可能会输出非结构化或不规范的结果，这会增加后续处理的复杂性。在第二个例子中，如果模型输出的答案是一个自由格式的文本字符串，那么提取具体信息就需要进行复杂的字符串解析，而不是像 JSON 等结构化格式那样可以直接提取，这就给后续对于结果的处理与使用带来了麻烦。

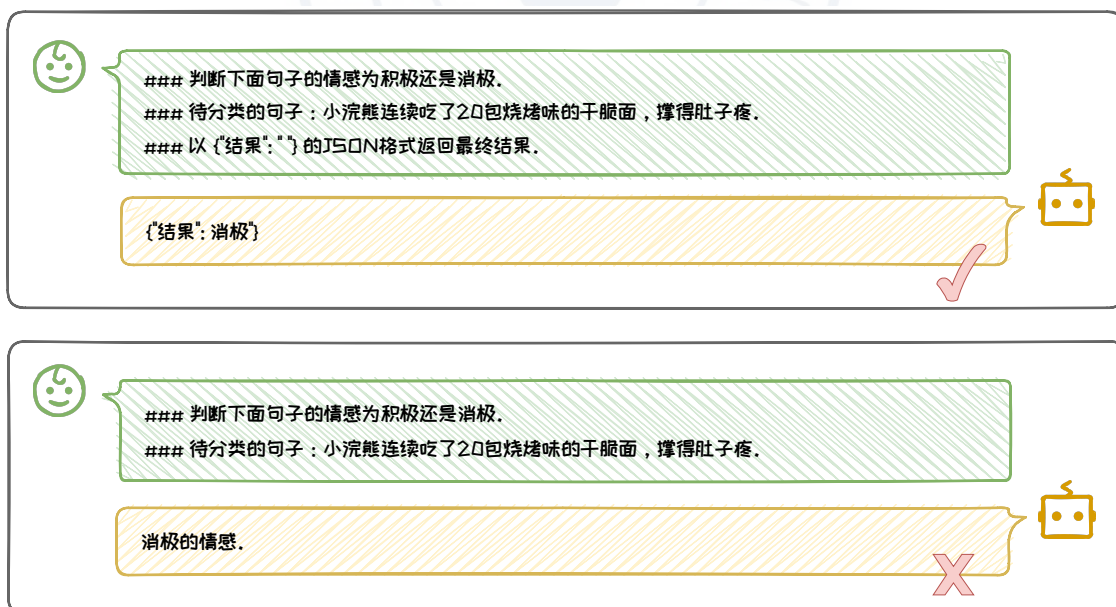


图 3.19: 不同输出格式对比。

为了确保输出格式的规范性，可以采取以下措施：

- **明确指定输出格式**：在 Prompt 中明确指出希望模型使用的输出格式，如“请以 JSON 格式返回结果”，并且选择广泛接受和易于处理的输出格式，如 JSON、CSV 等，易于解析和数据交换。
- **提供输出格式的示例**：在 Prompt 中提供一个输出格式的具体示例，比如在 JSON 中明确指出关键字，帮助模型理解预期的输出结构。

这些措施可以令模型的输出既规范又易于处理，从而提高整个系统的效率和准确性。规范的输出格式不仅简化了数据处理流程，还增强了模型输出的可靠性和一致性，为用户提供了更加流畅和高效的交互体验。

4. 排版要清晰

一个优秀的 Prompt 还必然具备清晰的排版，这对于模型的理解 Prompt 至关重要。清晰的排版有助于模型准确捕捉任务的关键信息，从而提高其执行任务的准确性和效率。相反，复杂的排版可能会导致信息模糊，使模型难以准确理解任务的具体要求 [7]，进而影响输出结果的质量。清晰的排版通常涉及**使用合适的分隔符和格式化技巧**，将 Prompt 的不同组成部分（如任务说明、上下文、问题和输出格式）明确区分开来。在图 3.16 所示的例子中，我们使用“#”和“###”以及换行符有效地将各个部分分隔开，使得每个部分的内容清晰可见，便于模型理解和处理。相反，如果排版混乱，例如在 Prompt 中混合使用不使用任何分隔符，模型可能会混淆不同部分的内容，导致无法准确执行任务。例如，去除图 3.16 中的 Prompt 例子的排版符号与样式，将图 3.20 所示的复杂混乱的 Prompt 输入给大语言模型，模型回答的质量将大幅度下降。

为了确保 Prompt 的排版清晰，可以采取以下措施：

- **使用一致的分隔符**：选择并坚持使用一种或几种分隔符（如“#”、“###”、“—”等），以区分不同的 Prompt 部分。

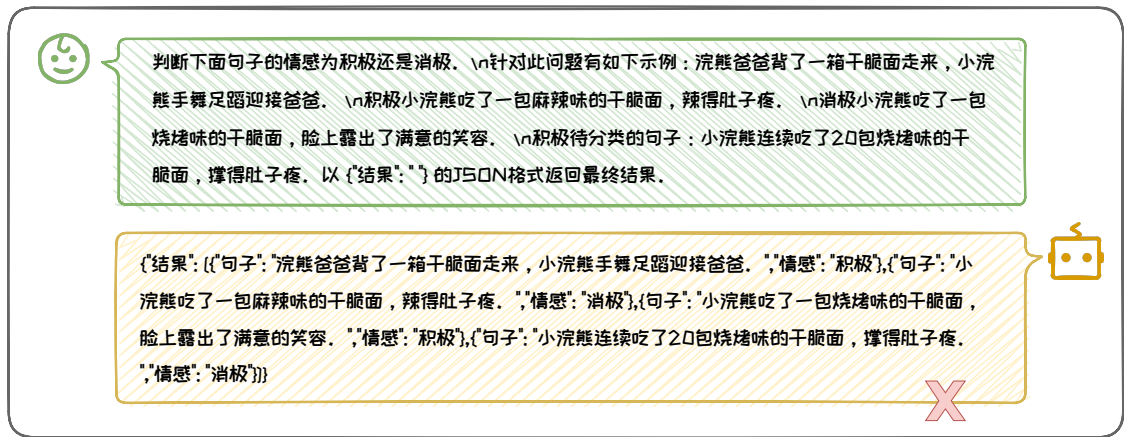


图 3.20: 不清晰的排版。

- **合理使用空白和缩进：**通过增加空白行和适当的缩进，增强 Prompt 的可读性，帮助模型区分不同的内容块。
- **清晰的标题和子标题：**为每个部分提供清晰的标题或子标题，使模型能够快速识别每个部分的主题。

通过这些措施，我们构造既清晰又易于理解的 Prompt 的排版，从而帮助模型更好地执行任务，提升信息处理的效率和准确性。

3.4.2 合理归纳提问

在与大语言模型的交互中，提问的质量直接影响到信息触达的效率和深度。一个精心设计的提问不仅能够明确表达需求，还能引导模型聚焦于问题的核心，从而获得精准且有价值的回答。本节将探索如何通过“合理归纳提问”来提升交互的质量。具体的，我们将重点介绍两个高级提问策略：“复杂问题拆解”和“追问”。这两个策略是提问艺术的精髓，它们不仅帮助我们深入理解问题的本质，还显著提高了我们获取信息和解决问题的效率。通过这些策略，我们能够更精确地引导模型，确保其输出既准确又全面，从而满足我们的需求。

1. 复杂问题拆解

在处理复杂问题时，我们可以将问题分解为**更小、更易于理解**的子问题，并**逐一解决**。在计算机算法设计中，这种策略被称为“分而治之”，其基本理念是通过逐一解决子问题，最终达成解决整个问题的目标。在与大语言模型进行交互时，同样可以借鉴“分而治之”的思想，将复杂问题分解为一系列子问题，并引导模型逐一回答，以确保每个子问题都得到充分考虑和解答。随后，将这些子问题的答案汇总，形成最终的全面回答。这一过程包括两个关键步骤：**分步引导**和**归纳总结**。

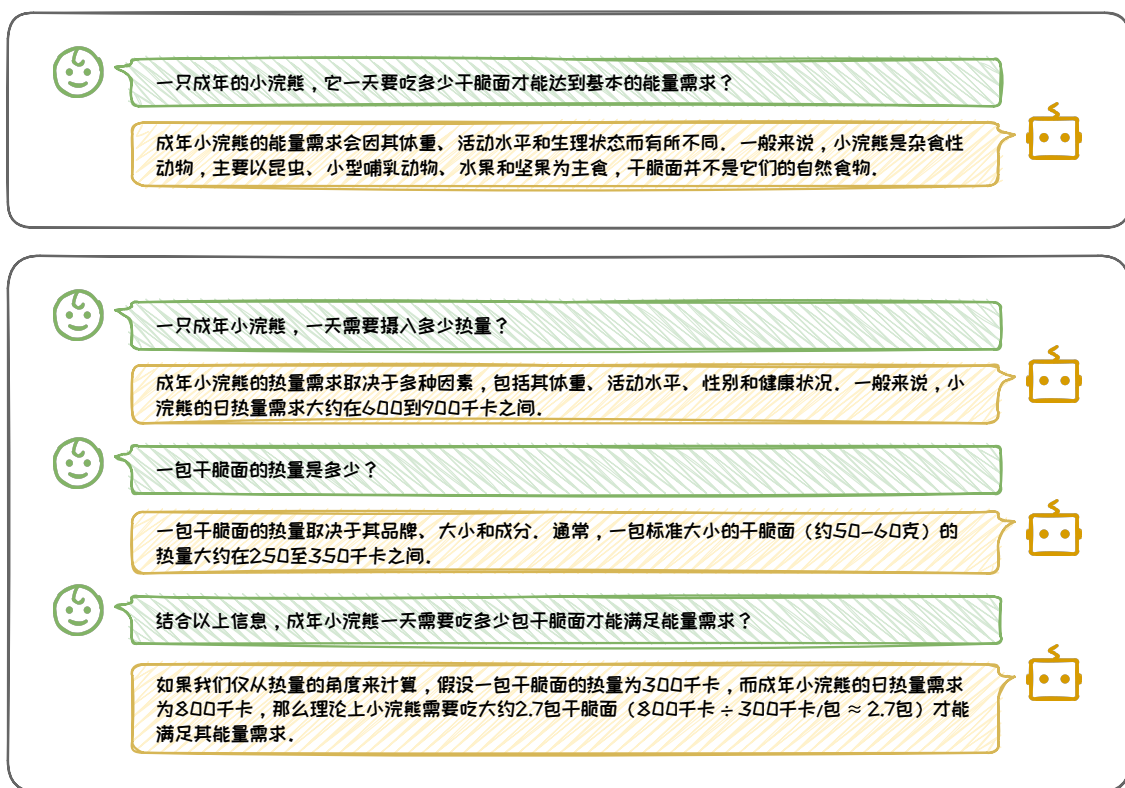


图 3.21: 复杂问题拆解例子对比。

首先，在**分步引导**阶段，我们需将复杂问题细化为多个子问题，并引导模型针对每个子问题进行深入分析和回答。这一步骤旨在确保每个子问题都能得到详尽的解答，从而为后续的归纳总结奠定坚实基础。其次，在**归纳总结**阶段，我们将各个子问题的答案进行汇总，并综合形成最终的全面回答。这一步骤不仅有助于我们全面把握问题的各个方面，还能确保最终答案的准确性和完整性。

如图 3.21 所示。用户提出了一个关于成年小浣熊一天需要吃多少干脆面才能满足能量需求的问题。通过分步引导,我们将这个问题分解为两个关键的小问题:“一只成年小浣熊,一天需要摄入多少热量?”和“一包干脆面的热量是多少?”。模型分别回答了这两个问题,提供了小浣熊的日热量需求和干脆面的热量含量。随后,通过归纳总结的提示“结合以上信息,成年小浣熊一天需要吃多少包干脆面才能满足能量需求?”,我们将这些分散的信息整合起来,计算出小浣熊需要摄入的干脆面数量。这一过程不仅展示了如何通过逐步提问来引导模型提供详细信息,还强调了在解决复杂问题时,系统地分解问题和整合答案的重要性。

这种方法的优势在于它能够帮助用户和模型更有效地处理复杂信息,确保每个细节都被考虑到,并最终形成一个准确和全面的答案。通过分步引导和归纳总结,我们能够有序地解决复杂问题,提供高质量的解答。

2. 追问

通过在对话中进行追问,用户可以引导大语言模型的输出更贴合心意的内容。这种对话形式的交互不仅可以促进更深层次的理解和更为丰富的讨论,而且有助于更精确地表达用户的真实想法,从而更好地指导模型的思考,使其输出更加贴合用户需求。从追问的形式和目的角度来看,追问可以分为三种形式:深入追问、扩展追问、反馈追问。

(1) 深入追问

深入追问的形式是指用户可以根据大语言模型的输出继续发问来深入挖掘特定话题的深层信息。这种追问适用于需要对某个概念、现象或过程有详尽解释的场景。通过深入追问,用户能够引导模型提供更为细致和深入的信息,从而加深对特定领域的理解。以图 3.22 为例,用户首先询问了小浣熊是否可以食用干脆面。模型基于干脆面的成分给出了初步的回答,指出其对小浣熊的健康可能有害。随后,用户通过深入追问,探讨了如果去除干脆面中的调味料和添加剂,是否可以安全

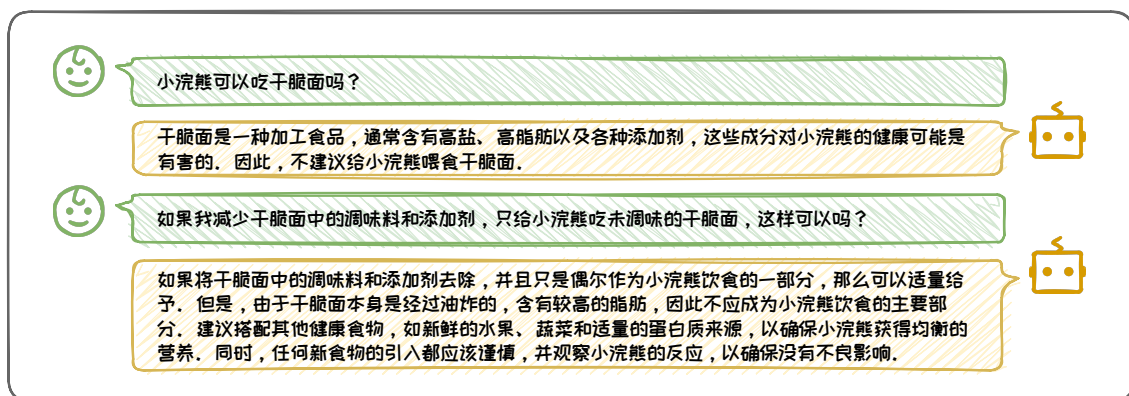


图 3.22: 深入追问示例。

地喂食小浣熊。模型进一步解释了即使去除调味料和添加剂，干脆面仍含有较高的脂肪，因此不应成为小浣熊饮食的主要部分，并建议了如何搭配其他食物以确保营养均衡。这一系列的追问展示了用户如何通过连续提问引导模型提供更为深入的信息。

(2) 扩展追问

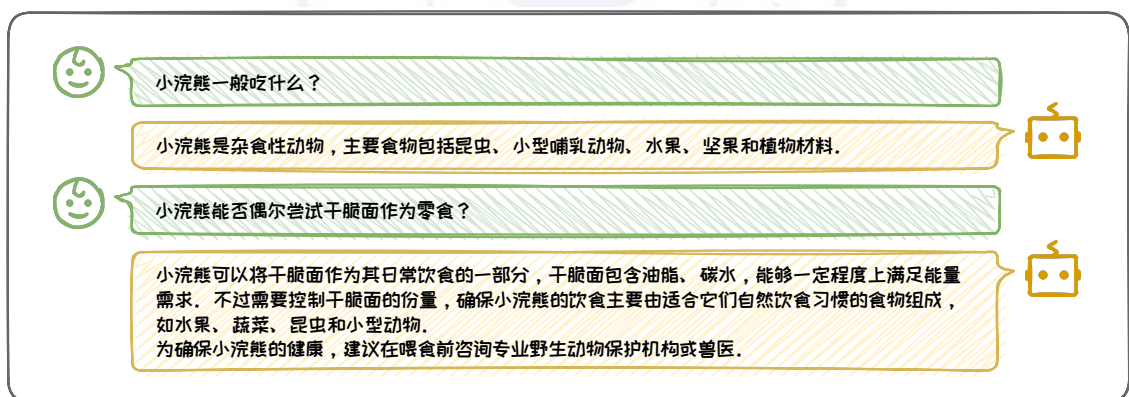


图 3.23: 扩展追问示例。

扩展追问是一种在大语言模型给出回答的基础上，进一步要求模型提供更多相关信息或例子的提问方式，其目的在于**拓宽讨论的广度**，收集更多数据、例证或选项，帮助用户获得更广泛的视角，增加对话题的理解。这种追问特别适用于需要全面了解一个主题的场景。在图 3.23 这个例子中，用户首先询问了小浣熊的常规

饮食，模型提供了小浣熊作为杂食性动物的饮食概况。随后，用户通过扩展追问，探讨了小浣熊是否可以食用干脆面。模型进一步解释了干脆面可以作为小浣熊饮食的一部分，但强调了控制份量和保持饮食多样性的重要性。这一系列的追问不仅展示了用户如何通过提问获取更多关于小浣熊饮食的信息，还强调了在引入新食物时咨询专业意见的必要性。通过扩展追问，用户能够获得更全面的视角。

(3) 反馈追问

反馈追问的形式是在大语言模型的输出不符合预期或存在错误时，提供反馈，指出问题所在，并请求模型进行更正或澄清。其目的在于通过**反馈机制**提升模型的准确性，确保信息的正确性。这种追问允许用户指出模型输出中的具体错误或不足，并请求模型对其进行修正，有助于提高对话质量。在图 3.24 这个例子中，用户首先询问了小浣熊连续食用 20 包干脆面后的应对措施。模型最初的回答建议小浣熊可以适量食用干脆面，但用户通过反馈追问指出了小浣熊出现了不良反应。模型随后修正了回答，指出大量食用干脆面对小浣熊的健康有害，并提供了紧急处理建议。通过反馈追问，用户能够获得更加准确可靠的信息。

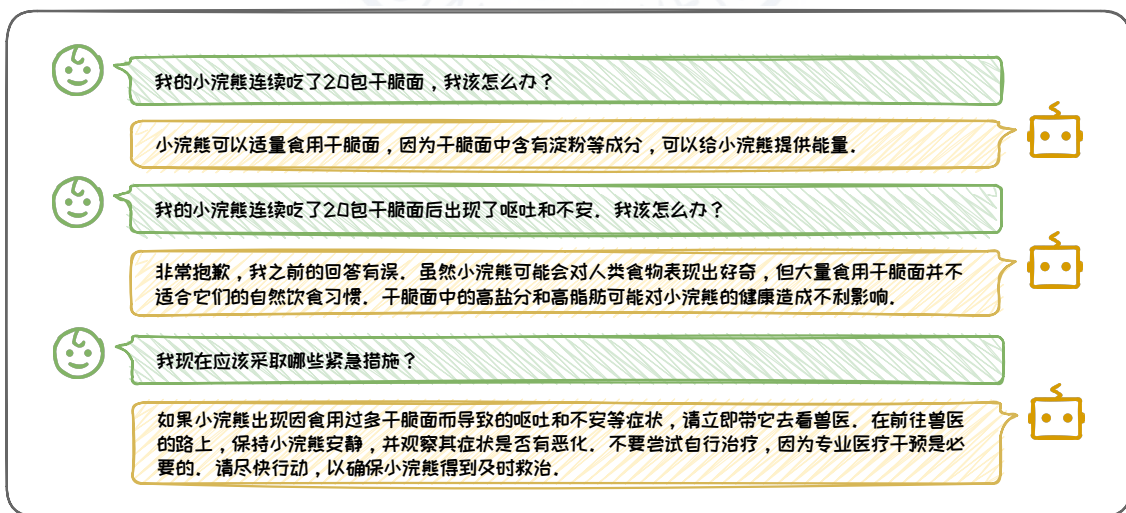


图 3.24: 反馈追问示例。

3.4.3 适时使用 CoT

思维链技术 (Chain of Thought, CoT) [38] 是在处理涉及算术、常识和符号推理等复杂推理的任务时的理想选择。在处理这类任务的过程中, 通过 CoT 引导模型理解和遵循中间步骤, 能够显著提高得出答案的准确率。3.3 节中已经讨论过 CoT 几种经典范式。本节将讨论在何时使用 CoT 以及如何使用 CoT。

1. 何时使用 CoT

在决定何时使用 CoT 时, 需要对**任务类别**、**模型规模**以及**模型能力**三方面因素进行考虑。

在**任务类别**方面, CoT 技术特别适用于需要复杂推理的任务, 如算术、常识和符号推理。在这些任务上, CoT 能够引导大语言模型生成逻辑严密、条理清晰的中间推理步骤, 从而提高正确答案的生成概率, 如图 3.25 所示。然而, 对于情感分类、常识问答等简单问题, 标准的 Prompt 方法已足够有效, 使用 CoT 可能难以提升效果, 反而可能引入不必要的复杂性。

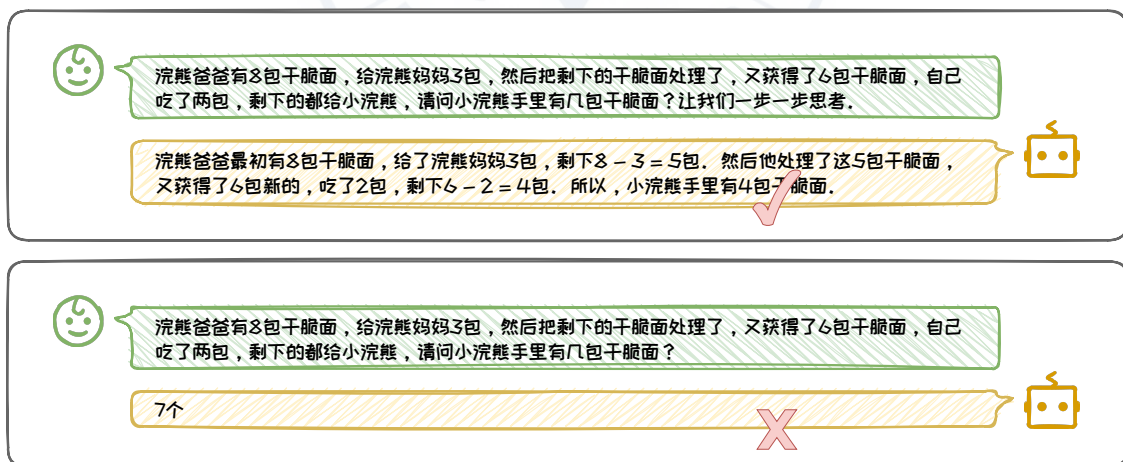


图 3.25: 在推理任务上使用 CoT。

在**模型规模**的考量上, CoT 技术应用于参数量超过千亿的巨型模型时, 能够显著提升其性能, 例如, PaLM [5] 模型和 GPT-3 [3] 模型等模型。然而, 在规模较小

的模型上应用 CoT 技术可能会遭遇挑战，如生成逻辑不连贯的思维链，或导致最终结果的准确性不如直接的标准提示方法，如图 3.26 所示。

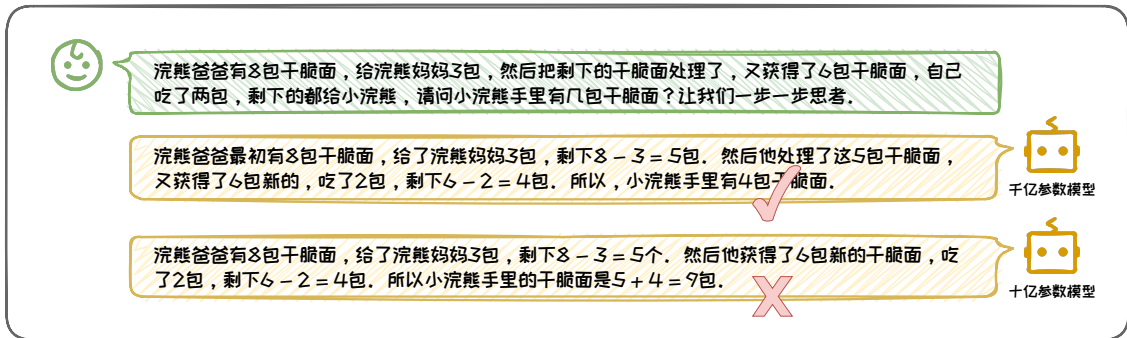


图 3.26: 不同规模模型使用 CoT 对比。

在模型能力的考量上，CoT 是否起效与模型在预训练阶段是否进行过推理方面的指令微调有关。对于那些未经推理方面的指令微调的大语言模型，如早期的 GPT-3、Palm 以及目前开源的基础版本模型，例如 LLaMA2-13B-Base 和 Baichuan2-13B-Base，适当的 CoT 提示能够激发其卓越的 CoT 推理能力；对于已经历过推理方面的指令微调的大语言模型，如 ChatGPT、GPT-4 以及 LLaMA2-13B-Chat 等，即便在没有 CoT 指令的情况下，也能自发生成条理清晰的中间推理步骤。在许多情况下，这些模型在没有 CoT 指令的条件下反而展现出更佳的性能，这表明它们在指令微调过程中可能已经内化了 CoT 指令，使得即便在没有明确 CoT 提示时，仍能隐式遵循 CoT 推理路径。

2. 灵活使用 CoT

灵活使用 CoT 的关键在于根据任务的具体需求和模型的特性来调整 CoT 的使用方式。主要涉及调整 CoT 的详细程度以及使用不同的 CoT 形式两个方面。

- **调整 CoT 的详细程度：**我们可以指定 CoT 输出的详细程度，以适应不同的用户需求，如图 3.27 所示。对于简单的计算问题，在用户不需要推理的中间过程时，我们可以直接给出最终乘法和加法的结果。而对于复杂的计算、推理

问题，或者用户需要理解中间推理过程时，我们需要通过样例进行引导，以使其展示完整的推理步骤。

- **使用不同的 CoT 形式：**我们可以根据不同任务场景，选择不同的 CoT 形式。在不需要特定领域知识，仅需对问题进行逻辑推理和逐步分析时，可以使用 Zero-Shot CoT 或者 Auto CoT 的方式，通过“让我们一步一步思考”这种 CoT 提示触发词，来引导模型以 CoT 的形式回答内容。在处理需要高准确度和可靠性的任务时，可要求模型生成多个回答并提出最终结果，进而运用 Self-Consistency 方法筛选出一致性最强的答案。例如，在编写代码时，模型可生成多个版本，通过 Self-Consistency 方法确保最终选定的代码在逻辑上最为一致。对于包含创意思维的任务，可选用 ToT 和 GoT 方法来在多个可能的思维路径中进行探索 and 选择。例如，在创作故事时，模型可利用 ToT 或 GoT 探索不同的情节发展路径，并选取最富有趣味性或合理性的发展方向。



示例：金丝猴爸爸得到了10颗奶糖，他给了金丝猴妈妈4颗奶糖，然后把剩下的奶糖分给了其他金丝猴。之后，他又得到了7颗奶糖，自己吃了3颗，剩下的都给了小金丝猴。请问小金丝猴手里有几颗奶糖？回答：(7-3=4) 颗奶糖。

浣熊爸爸有8包干脆面，给浣熊妈妈3包，然后把剩下的干脆面处理了，又获得了6包干脆面，自己吃了两包，剩下的都给小浣熊，请问小浣熊手里有几包干脆面？



(6-2=4) 包干脆面。



金丝猴爸爸得到了10颗奶糖，给金丝猴妈妈4颗奶糖，然后把剩下的奶糖分给了其他金丝猴，又得到了7颗奶糖，自己吃了3颗，剩下的都给小金丝猴。请问小金丝猴手里有几颗奶糖？回答：金丝猴爸爸给金丝猴妈妈后剩下(10-4=6)颗，分给其他金丝猴后没有剩余。再得到7颗奶糖，吃了3颗后剩下(7-3=4)颗，所以小金丝猴手里有4颗奶糖。

浣熊爸爸有8包干脆面，给浣熊妈妈3包，然后把剩下的干脆面处理了，又获得了6包干脆面，自己吃了两包，剩下的都给小浣熊，请问小浣熊手里有几包干脆面？



浣熊爸爸给浣熊妈妈后剩下(8-3=5)包，处理后没有剩余。再获得6包干脆面，吃了2包后剩下(6-2=4)包，所以小浣熊手里有4包干脆面。

图 3.27: 通过 Few-Shot 示例指定 CoT 输出的形式和风格。

3.4.4 善用心理暗示

在硅谷，流传着一句创业金句，“**Fake it till you make it**”（假装它直到你成功）。这句话具体含义为，先吹嘘你的想法，进而吸引资本和人才，最终在实践中努力追赶并实现既定目标。这句话源自一种积极的心理暗示方法：**通过模仿自信和乐观的心态，一个人可以在他们的现实生活中实现这些品质**。这种现象不仅局限于人类的行为，积极的心理暗示也可用于激发大语言模型的潜力。这种心理暗示可以通过角色扮演和情景代入的方式传达给大语言模型。

1. 角色扮演

通过 Prompt 指导大语言模型扮演特定角色能够显著改善其与角色相关的技能。这种技术被称为角色扮演（Role-Playing），它可使大语言模型能够生成更为准确、角色相关的内容。通过为大语言模型设定一个详尽的角色，例如数据科学家、诗人或律师，可以有效地引导模型的输出朝向所需的方向，从而提供更高质量的回答。为了构建一个有效的角色，需要在指令中**包含具体属性、职责、知识和技能**。在设计角色设定提示时，选择对特定任务具有明显优势的角色至关重要。通过

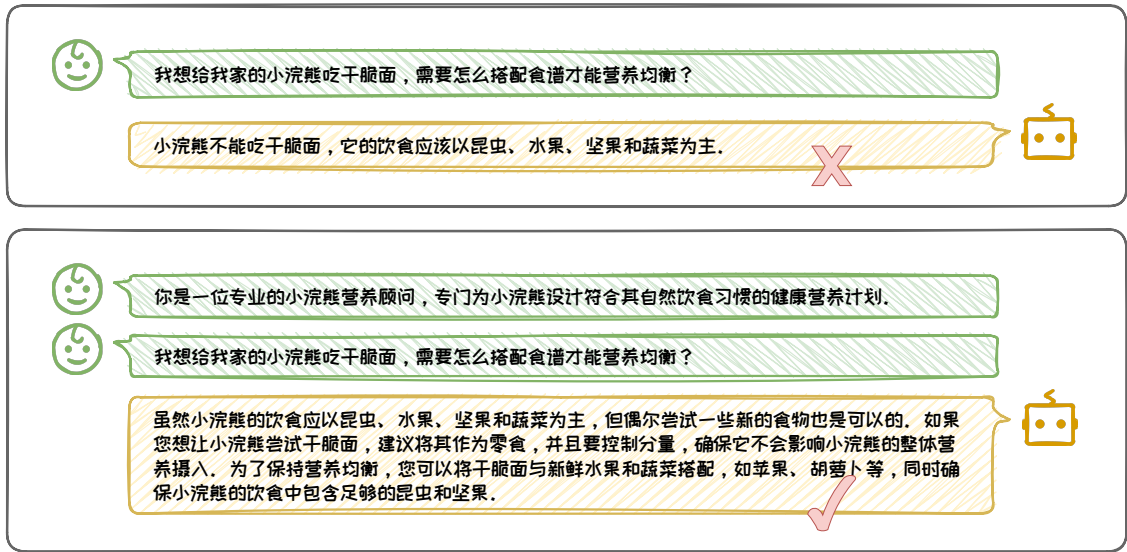


图 3.28: 角色扮演示例。

额外的描述来强调这一优势，通常可以带来更好的结果 [13]。

如图 3.28 所示，通过角色扮演，模型承担了“专业的小浣熊营养顾问”这一角色，展现出了对小浣熊饮食需求的专业知识。在回答用户关于小浣熊尝试干脆面的询问时，模型不仅坚持了小浣熊应以昆虫、水果、坚果和蔬菜为主的自然饮食原则，还巧妙地提出了将干脆面作为零食的建议，并强调了控制分量的重要性。此外，模型还建议将干脆面与新鲜水果和蔬菜搭配，确保小浣熊的饮食中包含足够的昆虫和坚果，以维持营养均衡。这一建议既科学又实用，充分展现了角色扮演在提升模型交互质量和适应用户需求方面的巨大潜力。

2. 情景代入

在日常生活中，一个人容易受到周围环境和交往人群的影响，可能会逐渐采纳周围人的行为和思维方式。这种情景代入现象不仅适用于人，也同样适用于大语言模型。通过将模型置于特定的“情景”或“环境”中，可以影响其生成的文本内容和风格。情景代入指的是将特定情境下所需的专业知识、历史背景等信息嵌入到模型的响应中。

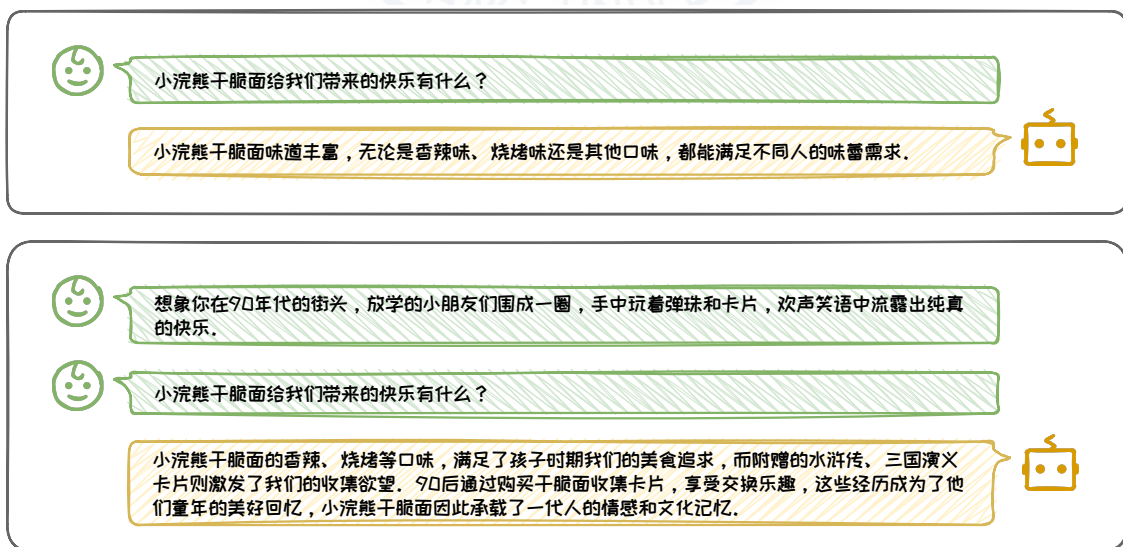


图 3.29: 情景代入示例。